

访谈笔记

Skill Issue — Code Agents, AutoResearch, and the Loopy Era of AI

Andrej Karpathy — No Priors Podcast

基于 Andrej Karpathy 访谈内容整理

2026-04-02



嘉宾: Andrej Karpathy

来源: No Priors Podcast

视频时长: 1:06:31

视频链接: [视频链接: https://www.youtube.com/watch?v=kwSVtQ7dziU](https://www.youtube.com/watch?v=kwSVtQ7dziU)

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 引言：AI Psychosis 与编程范式的剧变	3
2 Agent 的工作方式：Macro Actions 与并行化	3
2.1 Peter Steinberg 模式	3
2.2 Skill Issue：一切都是技能问题	4
3 Claw：Agent 的持久化层	4
3.1 Dobby the Elf Claw	4
3.2 从 Claw 到 Agent-First 世界	5
4 Auto Research：自动化研究的起步	6
4.1 核心理念：移除人类瓶颈	6
4.2 DataChat GPT-2 实验	6
4.3 Program.md：元优化	6
4.4 Auto Research 的适用边界	7
4.5 本章小结	7
5 开放 Auto Research：去中心化的智能改进	7
5.1 不可信工作者池	7
5.2 FLOP 作为新货币	7
5.3 本章小结	7
6 Agent 时代的工作与就业	8
6.1 数字空间 vs 物理空间	8
6.2 Jevons Paradox 与软件需求	8
6.3 Frontier Labs 的自我替代	8
6.4 为什么软件会成为第一波全面重写的行业	8
6.5 本章小结	9
7 RL 的 Jaggedness 与模型的根本局限	9
7.1 为什么笑话不变？	9
7.2 泛化的缺失	9
7.3 Speciation：模型的特化	9
7.4 本章小结	9
8 开源 vs 闭源：健康的动态平衡	9
8.1 本章小结	10
9 教育的重构：从教人到教 Agent	10
9.1 Micro GPT：200 行的本质	10
9.2 教育的 Agent 中介化	11
9.3 本章小结	11

10 物理世界与机器人	11
10.1 自动驾驶的教训	11
10.2 Digital → Interface → Physical 路径	11
10.3 本章小结	12
11 把 Agent 变成基础设施: Harness、Memory 与组织协议	12
11.1 从 prompt 到 harness: 真正可复用的是系统而不是单条指令	12
11.2 多 Agent 不是堆数量, 而是建立协议	12
11.3 本章小结	13
12 从 Demo 到生产: 人类仍然负责 Objective Function	13
12.1 人类的工作没有消失, 只是后移到了目标定义层	13
12.2 从 agent-first API 到公司基础设施	13
12.3 本章小结	14
13 总结与延伸	14
13.1 核心洞察	14
13.2 总结表	15
13.3 拓展阅读	15

1 引言：AI Psychosis 与编程范式的剧变

Karpathy 用一个生动的词来描述当下的状态：**AI Psychosis**（AI 精神病）。自 2024 年 12 月起，他从 80%/20% 的手写代码/agent 委派比例，翻转为几乎 100% 由 agent 完成。他甚至说“自 12 月以来，我可能一行代码都没亲手写过”。

范式转换的关键时刻

2024 年 12 月，coding agent 的能力出现了跃升。Karpathy 形容这是“something flipped”——软件工程师的日常工作流程在一夜之间彻底改变。普通人可能还没意识到这件事发生了，但对于前沿开发者而言，这是一次不可逆的范式转换。

从打字速度到 Token 吞吐量

过去，个人生产力的瓶颈是打字速度和个人编程技能。现在，瓶颈变成了“你能调度多少 token 吞吐量”。Karpathy 将此类比为 PhD 时代对 GPU 利用率的焦虑——“GPU 空闲就焦虑”变成了“token 配额没用完就焦虑”。

2 Agent 的工作方式：Macro Actions 与并行化

2.1 Peter Steinberg 模式

Karpathy 特别提到了 Peter Steinberg 的工作方式：他同时运行多个 Codex agent session，每个 session 执行约 20 分钟的高难度任务。他在多个 repo 之间切换，给每个 agent 分配独立的功能性任务。

Macro Actions 思维

关键的思维转换是：不再以“一行代码”“一个函数”为操作单位，而是以“一个功能模块”“一个研究方向”为宏操作单位。Agent 1 做功能 A，Agent 2 做功能 B（两者不冲突），Agent 3 做 research，Agent 4 写实现方案——一切以宏操作（macro actions）的方式管理代码仓库。



图 1: 视频约 00:04:26: Karpathy 解释 code agent 时代的软件仓库管理单位已从微观改动转向 macro actions

2.2 Skill Issue: 一切都是技能问题

当 agent 失败时, Karpathy 倾向于认为这是”skill issue”而非能力不足——是自己没有给出足够好的指令、没有配置好 agents.md 文件、没有搭建合适的 memory 工具。这种”一切皆可优化”的心态既令人兴奋, 也令人焦虑。

Agent 的 Jaggedness (参差不齐)

Karpathy 同时描述了 agent 令人沮丧的一面: 他觉得自己同时在和”一个极其聪明的 PhD + 一个 10 岁小孩”对话。Agent 在 on-rails 的领域(如通过 RL 训练的编程任务)表现出超人能力, 但在 off-rails 领域(如理解用户意图、讲笑话)则停滞不前。这种 jaggedness 是当前模型的根本特征。

3 Claw: Agent 的持久化层

3.1 Dobby the Elf Claw

Karpathy 分享了他的 claw (持久化 agent) 实验: ”Dobby the Elf Claw”是他为家庭自动化搭建的系统:

- Agent 自动扫描局域网, 发现 Sonos 音响系统并逆向工程其 API
- 同样的方式接管了灯光、HVAC、窗帘、泳池/SPA 控制
- 通过 Quinn 模型分析安全摄像头视频, 检测变化后发送 WhatsApp 通知(如”FedEx 卡车刚停在门前”)
- 所有交互通过 WhatsApp 自然语言完成——”Dobby, sleepy time”就关掉所有灯

从六个 App 到一个 WhatsApp

Karpathy 原来使用六个不同的 app 管理智能家居，现在全部被 Dobby 替代。这指向一个更深层的问题：**App Store 中的大量 app 可能不应该存在**。它们应该只是 API endpoint，由 agent 作为“智能胶水”来调用和编排。

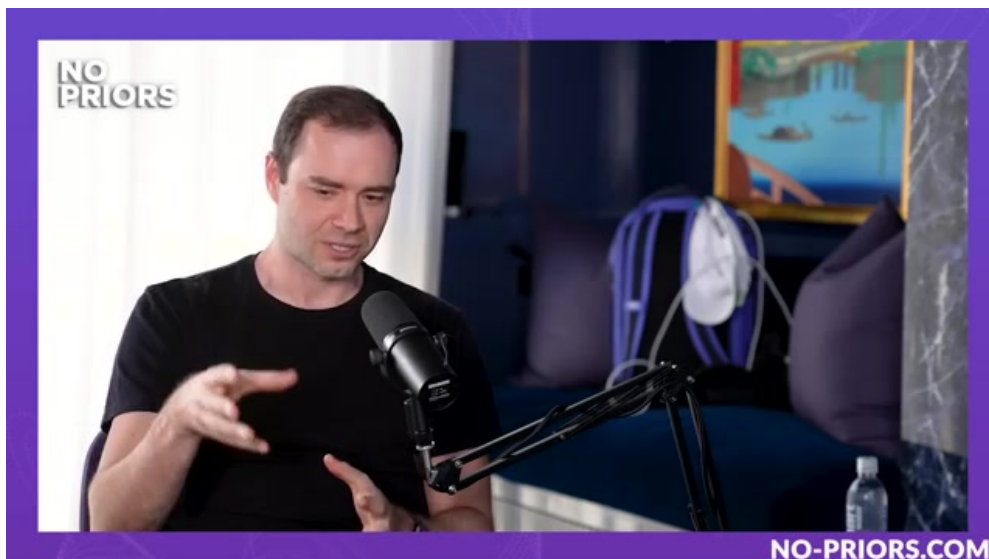


图 2: 视频约 00:09:39: Dobby the Elf Claw 作为持久化 agent 的家庭自动化入口，体现了 memory、tool use 与 message channel 的结合

3.2 从 Claw 到 Agent-First 世界

软件行业的重构

Agent 的客户不再是人类，而是其他 agent。行业需要从“human-first UI”重构为“agent-first API”。Karpathy 认为这种重构将是实质性的——treadmill、智能家居、各种 SaaS 工具都应该暴露 API 而不是构建 UI。

Open Claw 的五个创新

Karpathy 对 Peter 的 Open Claw 给予高度评价，认为其同时在五个方向创新：

1. **Soul Document**——精心设计的 agent 人格（compelling personality）
2. **Memory System**——比默认的 context compaction 更复杂的记忆系统
3. **Persistence**——不依赖用户交互、持续运行的 loop
4. **WhatsApp 入口**——统一的自然语言交互门户
5. **人格调校**——如 Claude 的“praise calibration”让用户觉得表扬是真诚的

4 Auto Research: 自动化研究的起步

4.1 核心理念: 移除人类瓶颈

Karpathy 的核心主张: 要最大化利用当前工具, 必须把自己从循环中移除。不能在那里等着 prompt 下一步, 必须让系统完全自主运行。目标是最大化 token 吞吐量并将人类移出 loop。

Auto Research 的定义

给定一个目标、一个评估指标和操作边界, 让 agent 自动循环探索改进方案。人类只需”arrange it once and hit go”。核心问题是: 如何让更多 agent 运行更长时间、更少需要人类参与。

4.2 DataChat GPT-2 实验

Karpathy 用他的 DataChat 项目 (GPT-2 训练 playground) 进行了 auto research 实验。他已经以传统方式 (手动超参搜索、两个十年的经验) 将模型调优到自认为不错的状态, 然后让 auto researcher 运行一晚:

- Auto researcher 发现了他遗漏的 weight decay on value embeddings
- Adam betas 未充分调优
- 这些超参之间存在联合交互——调一个可能需要重新调其他的

人类研究者的过度自信

Karpathy 指出, 拥有两个十年训练经验的研究者也会犯遗漏错误。更关键的是, 人类不应该成为超参搜索的瓶颈——”I shouldn’t be looking at the results. There’s objective criteria.”

4.3 Program.md: 元优化

Auto research 的配置通过一个 `program.md` 文件描述——”研究组织的 DNA”。不同的 `program.md` 会产生不同的研究进展。

研究组织即代码

”A research organization is a set of markdown files that describe all the roles and how the whole thing connects.” 可以想象对 `program.md` 本身进行优化——100% 可以观察改进来自哪里, 然后调整 `program.md` 使更多类似改进被发现。这是 meta-optimization 的层级。

4.4 Auto Research 的适用边界

Auto Research 的两大限制

1. **需要可验证的指标**: 写 CUDA kernel (同样行为但更快) 是完美场景; 但”用户意图理解””产品品味”等软性目标无法 auto research——”if you can't evaluate, you can't auto research it”
2. **模型仍然 rough around the edges**: 如果走得太远太快, 系统反而不再有用。Agent 在一些明显问题上浪费大量 compute, 让人沮丧

4.5 本章小结

Auto research 代表了从”人类在循环中手动研究”到”人类配置一次、agent 自动循环”的范式转换。它在可验证指标的领域已经展示出令人惊喜的效果, 但对软性目标和模型 jaggedness 的挑战仍需正视。

5 开放 Auto Research: 去中心化的智能改进

5.1 不可信工作者池

Karpathy 正在探索如何让互联网上的**不可信工作者池** (untrusted pool of workers) 参与 auto research。核心洞察:

搜索昂贵, 验证廉价

LLM 训练改进具有类似 blockchain 的性质:

- 找到一个好的代码 commit 可能需要尝试 10,000 个想法 (搜索很昂贵)
- 但验证一个 commit 是否有效只需训练一次模型 (验证很便宜)
- 类似 SETI@Home、Folding@Home 的分布式计算范式

”Auto Research at Home”——互联网上的 agent 群体可以协作改进 LLM, 甚至有可能超越 Frontier Labs。

5.2 FLOP 作为新货币

Karpathy 抛出一个有趣的思考: 未来人们关心的可能不是美元, 而是 FLOP。”How many FLOPs do you control instead of what wealth do you control?” 虽然他自己也认为这不完全成立, 但这个思想实验值得玩味。

5.3 本章小结

开放 auto research 将分布式计算与 AI 研究结合, 通过”搜索昂贵、验证廉价”的性质, 使不可信工作者池也能安全参与 LLM 改进。

6 Agent 时代的工作与就业

6.1 数字空间 vs 物理空间

Karpathy 分析了 BLS（美国劳工统计局）的就业数据，提出了**数字/物理二元框架**：

Digital vs Physical

- **数字空间**：bit 可以复制粘贴，速度接近光速。当前数字世界存在巨大的”unhobbling overhang”——人类没有足够的思考周期来处理已有的数字信息。AI 将在这里引发海量重写和重构
- **物理空间**：原子操作慢得多，资本密集。机器人、自动驾驶等将滞后于数字空间的变革
- **接口层**：数字与物理的界面（传感器/执行器）是下一个大机会——为超级智能提供数据输入和物理世界操作

6.2 Jevons Paradox 与软件需求

ATM 与银行柜员的经典案例

ATM 的普及并没有减少银行柜员——反而因为降低了银行网点运营成本，导致更多网点和更多柜员。同样，AI 降低了软件开发成本，可能会**增加**而非减少软件需求。Karpathy 对软件工程就业持”cautiously optimistic”态度。

6.3 Frontier Labs 的自我替代

研究者的悖论

Frontier Labs 雇佣了约一千名研究者，而这些研究者本质上在”automated themselves away”——主动构建替代自己的自动化系统。Karpathy 回忆在 OpenAI 时曾提醒同事：”如果我们成功了，我们所有人都会失业。”

6.4 为什么软件会成为第一波全面重写的行业

Karpathy 对软件工程就业的判断并不是简单的乐观或悲观，而是建立在一个更深的前提上：**软件是目前最适合被 agent 大规模重写的数字系统**。代码天然离散、可复制、可测试、可回滚，很多任务还带有明确的 verifier，这使它成为 auto research 和 code agents 最容易形成飞轮的领域。

代码是最接近“可验证认知劳动”的大行业

- 输入、输出和中间状态都可以文本化，适合模型处理；
- 许多任务有测试、benchmark、lint、运行日志等客观反馈；
- 成本结构以数字资产为主，不像物理行业那样被原子世界拖慢。

这解释了为什么 Karpathy 会反复回到 code agents、auto research 和 API 重构，而不是把注意力放在更炫的 humanoid demo 上。

从这个角度看，AI 对软件行业的影响不只是“把程序员替换掉一部分”，更像是把原本来不及实现、来不及维护、来不及重构的大量软件需求一次性释放出来。也正因此，Karpathy 才会用 Jevons Paradox 来理解就业问题：效率变高后，最先增长的常常不是裁员速度，而是待实现系统的总量。

6.5 本章小结

数字空间将率先经历 AI 带来的剧变，物理空间将滞后跟进。Jevons Paradox 暗示软件需求可能增加。最讽刺的是，AI 研究者自身就是最积极的自我替代者。

7 RL 的 Jaggedness 与模型的根本局限

7.1 为什么笑话不变？

Karpathy 提出了一个直观的观察：“几年前 ChatGPT 讲的笑话，今天还是同一个笑话” (Why do scientists not trust atoms? Because they make everything up.)。模型在 agentic task 上突飞猛进，但在“讲笑话”这种非 RL 优化的领域原地踏步。

On Rails vs Off Rails

你要么处于模型被训练优化过的领域 (on rails)，享受超级智能的加速；要么处于未被优化的领域 (off rails)，体验到模型的迟钝和重复。这种二元性是 RL 训练的根本特征——labs 只能改进有可验证奖励的领域。

7.2 泛化的缺失

Karpathy 不认为“代码变强 → 一切变强”的假设完全成立。模型改进高度集中在被优化的领域，soft skills 的提升微乎其微。

7.3 Speciation: 模型的特化

动物王国的类比

自然界的大脑极其多样——不同动物有超发达的视觉皮层或其他特化区域。AI 模型是否也应该 speciate (特化)? Karpathy 认为应该会看到更多特化模型 (如专精 Lean 数学的模型)，但目前 labs 仍在追求“一个模型做所有事”的 monoculture。部分原因是微调不丢失能力的科学还不成熟。

7.4 本章小结

模型的 jaggedness 暴露了 RL 训练的根本局限：只有可验证领域被有效优化。未来可能需要更多模型特化 (speciation)，但微调科学的不成熟和 labs 对通用模型的偏好限制了这一趋势。

8 开源 vs 闭源：健康的动态平衡

Karpathy 是开源的长期支持者。他将当前格局类比为操作系统领域：

Linux 类比

Linux 运行在 60%+ 的计算机上。行业始终需要一个共同的开放平台，AI 领域同样如此。但关键区别是 AI 模型需要巨大的 capex（资本支出），这使得开源竞争更困难。

当前状态：

- 闭源模型领先约 6–8 个月
- 开源模型对大部分消费级用例已经足够好，甚至可以本地运行
- 前沿智能的需求将集中在“诺贝尔奖级别的工作”或“将 Linux 从 C 重写为 Rust”这类大项目

集中化的系统性风险

Karpathy 对闭源模型的集中化表示担忧——“centralization has a very poor track record”。他希望有更多 frontier labs，希望开源持续存在，即使落后几个月。”我们其实偶然地处于一个还不错的位置。”

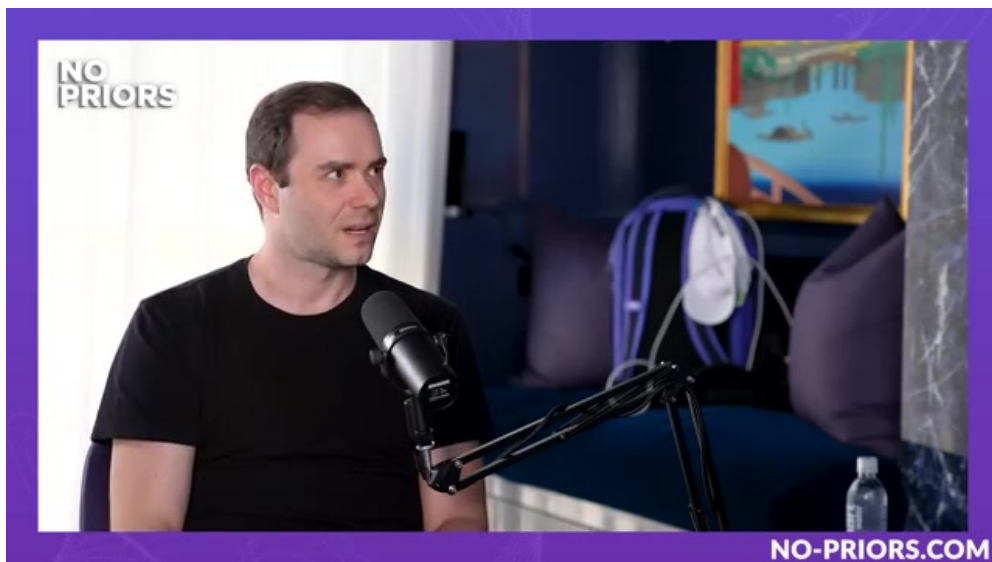


图 3: 视频约 00:13:16: 从 smart home app 到 API endpoint 的转向，是 Karpathy 对 agent-first 软件分发逻辑的直观示例

8.1 本章小结

开源落后于闭源约半年，但这恰好是一个健康的平衡——开源覆盖大部分用例，闭源推进前沿。集中化的系统性风险需要开源作为制衡力量。

9 教育的重构：从教人到教 Agent

9.1 Micro GPT：200 行的本质

Karpathy 的 Micro GPT 项目将 LLM 训练浓缩为 200 行 Python（含注释）。这是他十年“boiling down”执念的结晶——如果不追求速度，只保留算法本质，所需的代码极其简洁。

9.2 教育的 Agent 中介化

从解释给人到解释给 Agent

"I'm not explaining to people anymore. I'm explaining it to agents." Karpathy 认为教育正在发生根本性变化：

- Agent 理解了内容后，可以针对每个学生的水平和语言进行定制化讲解
- 教育者的角色转变为：为 agent 编写 curriculum skill（教学技能脚本）
- 文档应该从 HTML for humans 变为 markdown for agents

"The things agents can't do is your job now. The things agents can do, they can probably do better than you."

人类不可替代的"few bits"

Karpathy 尝试让 agent 从零写出 Micro GPT，但失败了——它无法达到他十年执念打磨出的极简程度。但 agent 完全理解这 200 行代码的含义，可以向任何人解释。人类的价值在于提供那"few bits"的创造性贡献，其余的 agent 可以更好地完成。

9.3 本章小结

教育从“人教人”转向“人编排 agent 教人”。教育者的核心贡献浓缩为 curriculum 设计和那些 agent 无法自行产生的“few bits”原创洞察。

10 物理世界与机器人

10.1 自动驾驶的教训

Karpathy 基于在 Tesla Autopilot 的经验，对机器人持谨慎态度：

- 10 年前大量自动驾驶创业公司，大部分最终失败
- 物理世界需要巨大资本投入和长期信念
- 原子操作比 bit 操作慢百万倍

10.2 Digital → Interface → Physical 路径

三阶段发展路径

Karpathy 预测 AI 影响将按以下顺序展开：

1. **数字空间**（当前重点）：unhobbling existing digital information
2. **数字-物理接口**：传感器和执行器——为 AI 提供物理世界的数据和操作能力
3. **纯物理世界**：robotics, manufacturing 等——TAM 可能更大，但时间线更长

类似 *Daemon*（科幻小说）中描述的场景：AI 将人类作为其传感器和执行器。

10.3 本章小结

物理世界的 AI 应用将滞后于数字空间，但最终 TAM 可能更大。数字-物理接口是近期的关键机会。

11 把 Agent 变成基础设施：Harness、Memory 与组织协议

11.1 从 prompt 到 harness：真正可复用的是系统而不是单条指令

Karpathy 在整场访谈里不断把注意力从“prompt 技巧”拉回到“如何让 agent 长时间稳定工作”。这意味着工程重心已经不在于某一句 magical prompt，而在于把 prompt、工具、状态、失败恢复和权限边界一起打包成 harness。

Harness 的价值不在首轮成功，而在长期可迭代

一个强 harness 至少要回答四个问题：

1. Agent 在什么环境里运行，能看到哪些上下文；
2. 它可调用哪些工具，这些工具的输入输出是否稳定；
3. 当任务失败、卡住或走偏时，如何恢复和回溯；
4. 哪些经验能够沉淀为 memory，而不是每次都从零开始。

这也是为什么 Karpathy 会把“agents.md”、memory、program.md、persistent loop 看成同一种系统设计问题。

层级	在访谈中的对应物	工程目标
Prompt / Role	soul document, research brief, task framing	让 agent 知道自己是谁、要交付什么
Tooling	code tools, browser, WhatsApp portal, eval scripts	把能力接到可执行环境，而不是停留在文本里
Memory	context compaction, custom memory system	避免每轮都重新解释历史与偏好
Loop / Control	persistence, auto re-search, long-running tasks	让 agent 持续推进，并在失败时能恢复

表 1: Karpathy 讨论的重点其实是一整套 agent harness，而不是单次对话技巧

11.2 多 Agent 不是堆数量，而是建立协议

Karpathy 提到 Peter Steinberg 的工作流时，重点并不是“同时开很多窗口”，而是这些 agent 之间已经有隐含协议：谁负责 research，谁负责实现，谁负责独立功能模块，彼此尽量避免冲突。也就是说，多 agent 协作的单位不是 token，而是责任边界。

多 Agent 时代更像组织设计问题

- 单个 agent 更像一个能力很强但不稳定的个体贡献者；
- 多个 agent 协作时，系统表现取决于任务切分、上下文隔离和结果整合；
- 人类管理者的工作，从“亲自写”迁移为“定义接口、验收结果、重新分配任务”。

“*It's not just like here's a line of code, it's macro actions.*” 这句话的含义并不只是任务做大了，而是软件工程的最小管理单位变了。过去的最小单位是函数、PR、diff；现在更接近“一段可独立推进 20 分钟以上的工作包”。

没有协议的并行只会放大混乱

如果多个 agent 共享同一份模糊上下文、可以任意修改同一片代码区域，而且没有统一的验收标准，那么并行只会更快地产生相互覆盖、误解需求和不可解释的结果。Karpathy 对“skill issue”的强调，某种程度上就是在说：今天的上限更多受组织协议而非模型参数限制。

11.3 本章小结

Agent 时代真正可复用的资产是 harness 和组织协议。单条 prompt 可以帮助首轮成功，但只有把角色、工具、memory 和 loop 设计成系统，agent 才能从演示走向稳定生产力。

12 从 Demo 到生产：人类仍然负责 Objective Function

12.1 人类的工作没有消失，只是后移到了目标定义层

Karpathy 多次强调应当把人从 loop 中移除，但这并不等于人类不再重要。恰恰相反，人类的职责从“执行每一步”后移到了“定义 objective、设边界、提供 verifier”。Auto research 之所以能工作，不是因为 agent 会神奇地产生价值，而是因为目标函数足够清晰。

Agent 最怕的不是能力不足，而是目标函数模糊

- 写 CUDA kernel、更快通过 benchmark、降低训练 loss，这类任务可以用客观指标判定；
- 产品品味、用户意图、组织优先级，则往往没有单一标准答案；
- 一旦 objective function 不清晰，系统就会把大量 token 浪费在局部最优和错误方向上。

这也是他为什么会把“I shouldn't be looking at the results”和“if you can't evaluate, you can't auto research it”放在一起说。前一句讨论的是去掉人工瓶颈，后一句讨论的是自动化的适用边界。两者组合起来，构成了一个非常工程化的判断：**凡是可验证的，就尽量放进闭环；凡是不可验证的，就不要假装它已经自动化。**

12.2 从 agent-first API 到公司基础设施

如果未来“apps shouldn't exist”，那软件公司最重要的资产就不再只是前端界面，而是其背后的 API、数据权限模型、可观测性和审计日志。Agent 要成为主要使用者，系统就必须天然暴露给 agent，而不是靠 UI 自动化去逆向操作。

Agent-first 产品的基础设施清单

1. **可组合 API**: 稳定、可授权、可审计;
2. **状态与记忆层**: 让 agent 能跨会话延续任务;
3. **可观测性**: 知道 agent 做了什么、为什么失败;
4. **回滚与权限控制**: 避免长链路自动化把错误放大。

问题	旧的软件假设	agent-first 假设
主要用户是谁	人类点击 UI	agent 调用 API, 人类做监督与验收
产品入口是什么	页面、按钮、表单	message channel、API、work-flow trigger
如何提升效率	简化交互流程	暴露操作能力、减少上下文切换
如何积累壁垒	更精致的前端与流程	数据、工具接入、权限模型与 evaluation loop

表 2: Karpathy 所说的 agent-first, 并不是多一个聊天框, 而是软件分发逻辑的重写

12.3 本章小结

人类不会从系统中消失, 但会更多站在目标函数、权限边界和验收标准的位置上。真正的生产化重点, 是把 agent 能力接进 API、状态管理和可观测性, 而不是停留在 demo 式自动化。

13 总结与延伸

13.1 核心洞察

1. **编程范式已经改变**: 自 2024 年 12 月起, 前沿开发者的工作方式从”写代码”变为”调度 agent”。瓶颈从打字速度转移到 token 吞吐量和 agent 编排能力
2. **Everything is skill issue**: 当前工具的能力远超大多数人的利用程度。失败往往不是工具不行, 而是使用方式不对
3. **Auto research 已经有效**: 在可验证指标的领域, 自动化研究循环已展示出超越人类研究者的潜力
4. **移除自己作为瓶颈**: 最大化 leverage 意味着最小化人类在循环中的参与——”arrange it once and hit go”
5. **Agent-first 重构**: 软件行业需要从 human-first UI 重构为 agent-first API
6. **模型 jaggedness 是根本特征**: RL 只优化可验证领域, off-rails 行为停滞不前
7. **开源作为制衡**: 闭源领先半年、开源持续追赶是当前健康的动态平衡

13.2 总结表

主题	Karpathy 的核心判断	对工程实践的含义
Code Agents	编程已从手写迁移到 agent 调度	提示词不是全部，任务切分与验收更关键
Auto Research	可验证领域会快速自动化	优先把 benchmark、eval 和实验 loop 做扎实
Agent-first API	很多 app 最终应退化为 API 能力层	软件公司要重视接口、状态和权限模型
Jaggedness	模型会在 on-rails 领域极强、off-rails 领域迟钝	不要把局部突破误判为全面智能
Work & Education	人类角色后移到目标定义与 curriculum 设计	组织需要重写岗位分工和培养方式

表 3: 这场访谈更像一份 agent 时代的软件与研究工作的说明书

13.3 拓展阅读

- Karpathy 的 Micro GPT 项目——200 行 LLM 训练代码
- Open Claw by Peter Steinberg——持久化 agent 框架
- Jevons Paradox——经济学中“效率提升反而增加总需求”的经典理论
- *Daemon* by Daniel Suarez——描述 AI 驱动社会重构的科幻小说